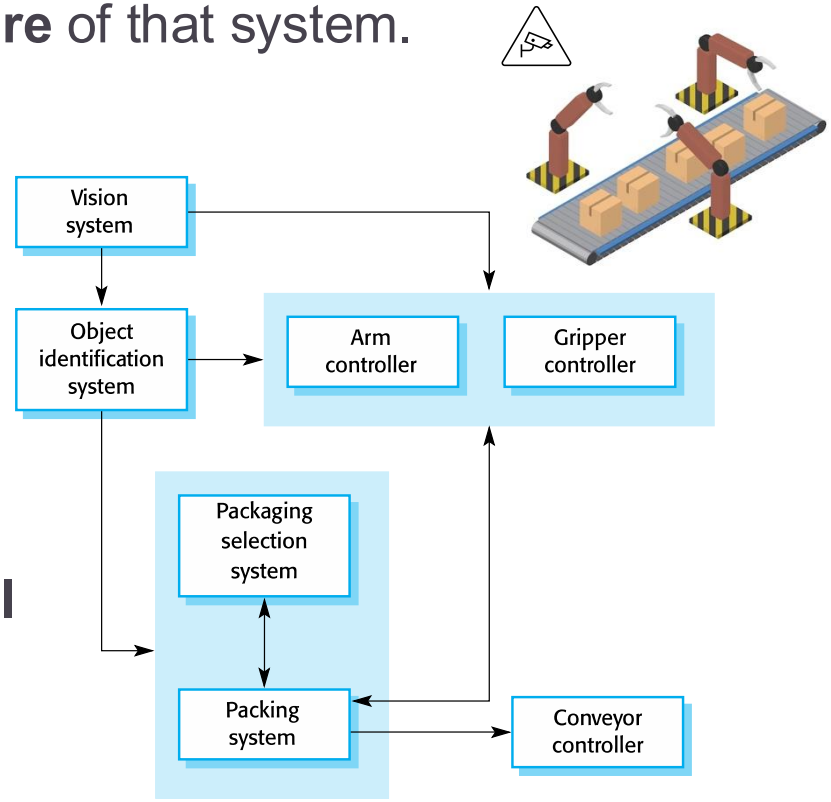
L5 – Design: Architecture, Patterns & SOLID

Topics covered

- ✧ Architectural design
  - Architectural design decisions
  - Architectural patterns
- ✧ SOLID design principles
- ✧ Design patterns

Architectural design

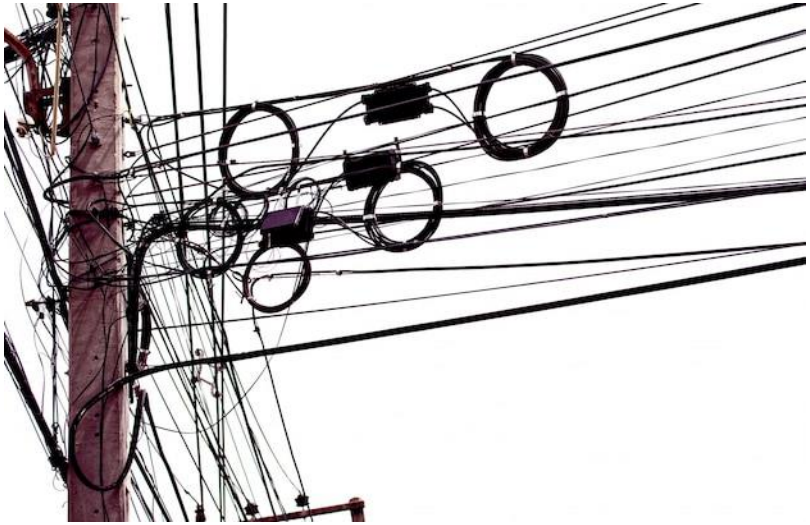
- Understanding how a software system should be organized and designing the **overall structure** of that system.
- Architectural design is the critical link between design and requirements engineering, as it **identifies the main structural components** in a system and the **relationships** between them.
- The output of the architectural design process is an **architectural model** that describes how the system is organized as a set of communicating components.



Example: The architecture model of a packing robot control system

Agility and architecture

- It is generally accepted that an **early stage of agile processes** is to design an overall systems architecture.
- **Refactoring the system architecture** is usually **expensive** because it affects so many components in the system



Architectural abstraction

✧ Architecture in the small

- is concerned with **the architecture of individual programs**. At this level, we are concerned with the way that an individual program is decomposed into components.

✧ Architecture in the large

- is concerned with **the architecture of complex enterprise systems** that include other systems, programs, and program components. These enterprise systems are distributed over different computers, which may be owned and managed by different companies.

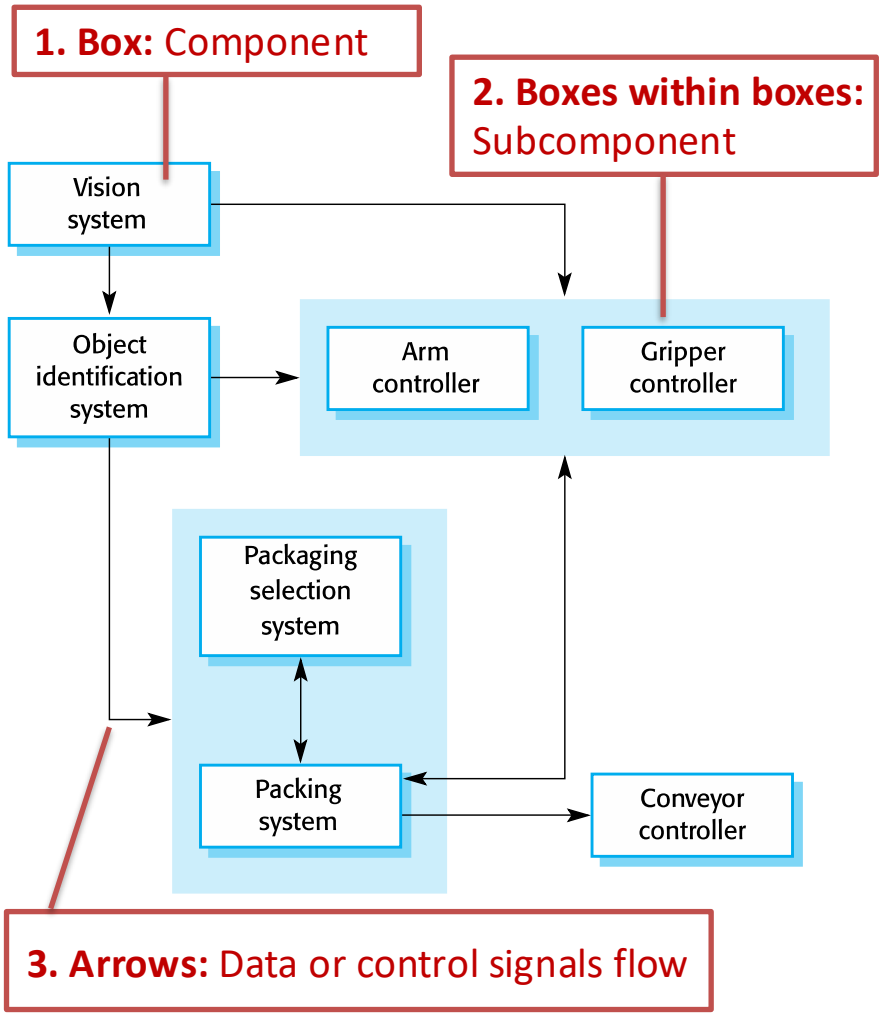


Advantages of explicit architecture

- **Stakeholder communication**
  - Architecture may be used as a focus of discussion by system stakeholders.
- **System analysis**
  - Means that analysis of whether the system can meet its **non-functional requirements** is possible.
- **Large-scale reuse**
  - The architecture may be reusable across a range of systems

Architectural representations

- **Box and line diagrams - Simple, informal block diagrams** showing entities and relationships
  - Very abstract
  - However, **useful for communication** with stakeholders and for project planning.
- But these have been **criticised** because they **lack semantics.**
- Use of architectural models
  - facilitating discussion
  - documenting an architecture

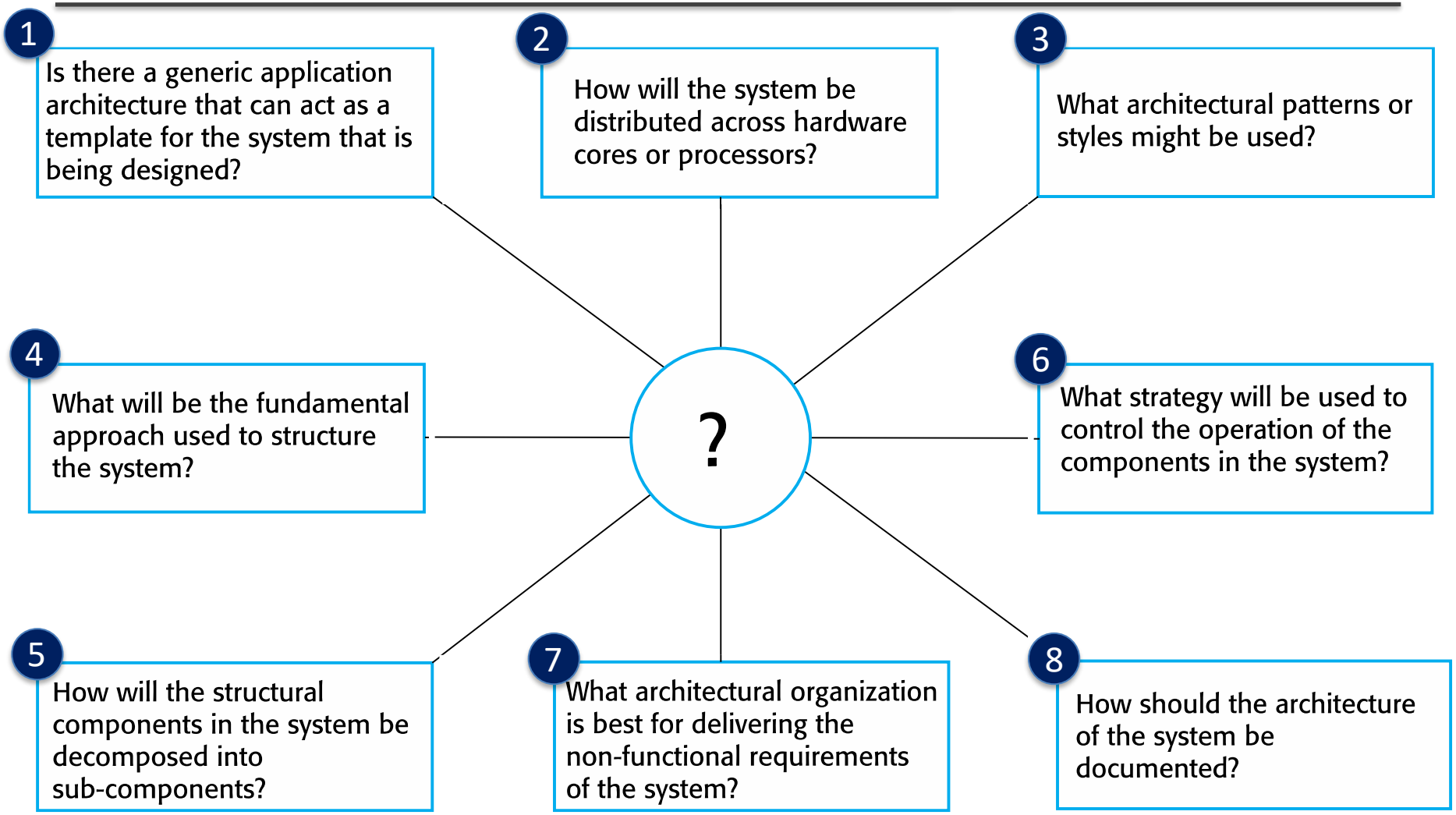


Architectural design decisions

Architectural design decisions

- Architectural design is a **creative process** so the process differs depending on the type of system being developed.
- However, **a number of common decisions** span all design processes and these decisions **affect the non-functional characteristics** of the system.

Architectural design decisions



Architecture reuse

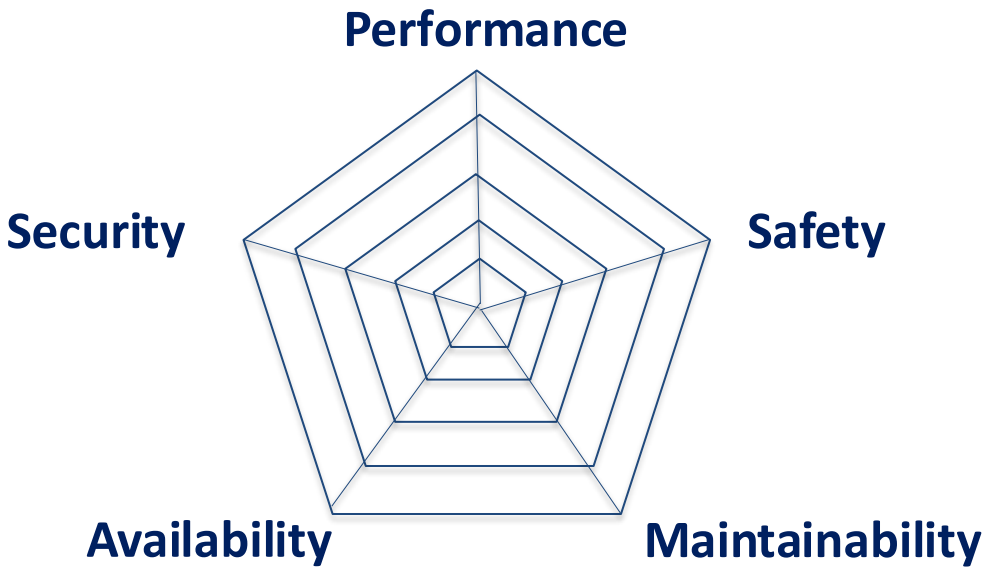
- Systems in the **same domain** often have **similar architectures** that reflect domain concepts.
- Application product lines are built around **a core architecture** with variants that satisfy particular customer requirements.
  - e.g. Microsoft 365
- The architecture of a system may be designed around one of more **architectural patterns or 'styles'.**
  - These capture the essence of an architecture and can be instantiated in different ways.

Architecture and system characteristics

- **Performance**
  - Localise critical operations and minimise communications. Use large rather than fine-grain components.
- **Security**
  - Use a layered architecture with critical assets in the inner layers.
- **Safety**
  - Localise safety-critical features in a small number of sub-systems.
- **Availability**
  - Include redundant components and mechanisms for fault tolerance.
- **Maintainability**
  - Use fine-grain, replaceable components.

Open Floor

Do you find any potential conflict to achieve these five system characteristics at the same time? Give an example.



Performance vs Maintainability through benchmark (1)

WithSingleLevel

```
[Benchmark]
[ArgumentsSource(nameof(Arrays))]
public void WithSingleLevel(int[] array)
{
    PerformOperationsWithSingleLevel(array);
}

private void PerformOperationsWithSingleLevel(int[] array)
{
    int[] filteredNumbers = array.Where(n => n % 12 != 0).ToArray();

    foreach (var number in filteredNumbers)
    {
        string status = "";
        var isOnDb = number % 3 == 0;
        if (isOnDb)
        {
            status = "onDB";
        }
        else
        {
            var isOnCache = (number + 1) % 7 == 0;
            if (isOnCache)
            {
                status = "onCache";
            }
            else
            {
                status = "toBeCreated";
            }
        }
    }
}
```

WithNestedLevels

```
[Benchmark]
[ArgumentsSource(nameof(Arrays))]
public void WithNestedLevels(int[] array)
{
    PerformOperationsWithMultipleLevels(array);
}

private void PerformOperationsWithMultipleLevels(int[] array)
{
    int[] filteredNumbers = GetFilteredNumbers(array);

    foreach (var number in filteredNumbers)
    {
        CalculateStatus(number);
    }
}

private static void CalculateStatus(int number)
{
    string status = "";
    var isOnDb = IsOnDb(number);
    status = isOnDb ? GetOnDBStatus() : GetNotOnDbStatus(number);
}

private static string GetNotOnDbStatus(int number)
{
    var isOnCache = IsOnCache(number);
    return isOnCache ? GetOnCacheStatus() : GetToBeCreatedStatus();
}

private static string GetToBeCreatedStatus() => "toBeCreated";

private static string GetOnCacheStatus() => "onCache";

private static bool IsOnCache(int number) => (number + 1) % 7 == 0;

private static string GetOnDBStatus() => "onDB";

private static bool IsOnDb(int number) => number % 3 == 0;

private static int[] GetFilteredNumbers(int[] array) => array.Where(n => n % 12 != 0).ToArray()
```

Performance vs Maintainability through benchmark (2)

Method	array	Mean	Error	StdDev	Median
WithSingleLevel	Int32[10000]	46,384.6 ns	773.95 ns	1,997.82 ns	45,605.9 ns
WithNestedLevels	Int32[10000]	58,912.2 ns	1,152.96 ns	1,539.16 ns	58,536.7 ns
WithSingleLevel	Int32[1000]	5,184.9 ns	100.54 ns	89.12 ns	5,160.7 ns
WithNestedLevels	Int32[1000]	6,557.1 ns	128.84 ns	153.37 ns	6,529.2 ns
WithSingleLevel	Int32[100]	781.0 ns	18.54 ns	51.99 ns	764.3 ns
WithNestedLevels	Int32[100]	910.5 ns	17.03 ns	31.98 ns	901.5 ns
WithSingleLevel	Int32[10]	186.7 ns	3.71 ns	9.43 ns	182.9 ns
WithNestedLevels	Int32[10]	193.5 ns	2.48 ns	2.07 ns	193.7 ns

Ref: code4it - clean code vs performance

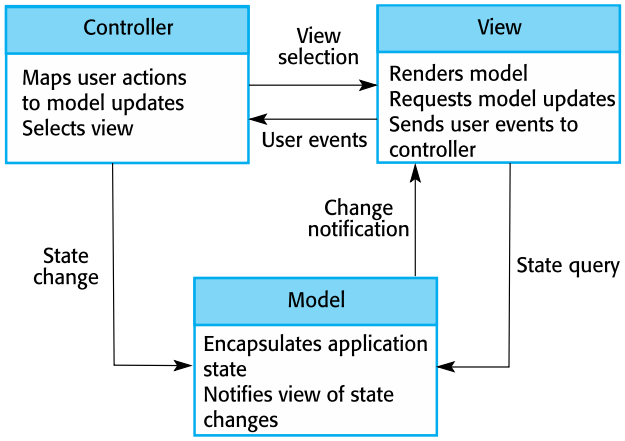
Architectural patterns

Architectural patterns

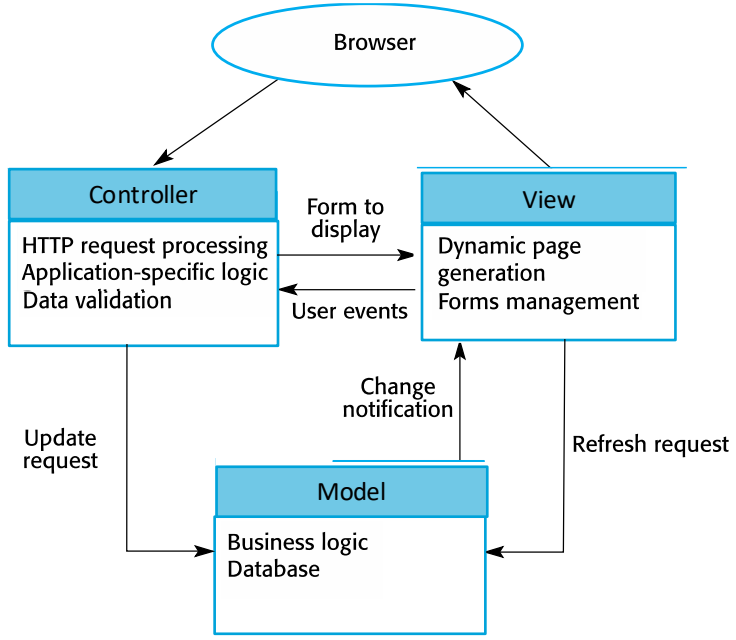
- Patterns are a means of **representing, sharing and reusing knowledge**.
- An architectural pattern is a stylized description of **good design practice**, which has been tried and tested in different environments.
- Patterns should include information about **when they are** and **when they are NOT useful**.

Model-View-Controller (MVC) pattern

- **Description**: Separates presentation and interaction from the system data. Structured into three logical components that interact with each other.
- **When used**: multiple ways to view and interact with data / future requirements for interaction and presentation of data are unknown
- **Advantage**: Allows the data to change independently of its representation and vice versa
- **Disadvantage**: involve additional code and code complexity



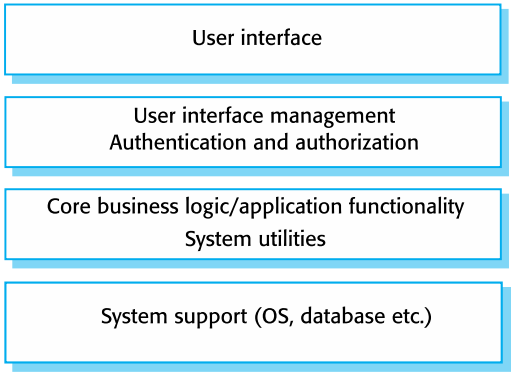
MVC Pattern



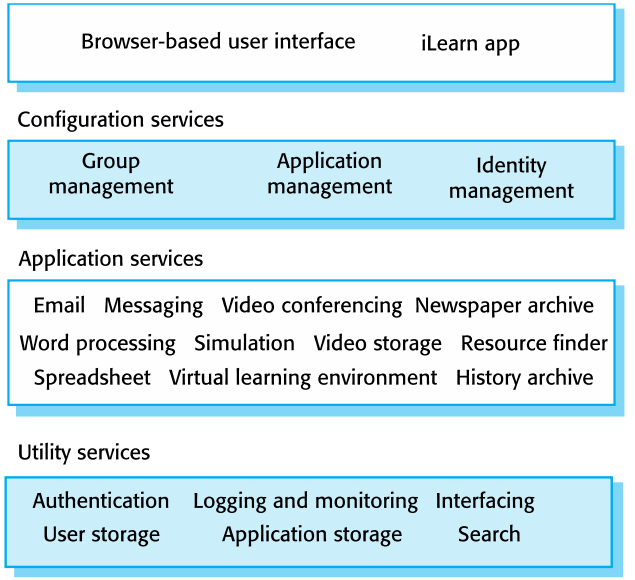
Example: Web application architecture using the MVC pattern

Layered architecture pattern

- **Description**: Organizes the system into layers with related functionality associated with each layer. Lowest-level layers represent core services.
- **When used**: Building new facilities on top of existing systems / each team responsibility for a layer of functionality / multi-level security
- **Advantages**: Allows replacement of entire layers so long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability
- **Disadvantages**: Clean separation between layers is often difficult. High-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance may be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.



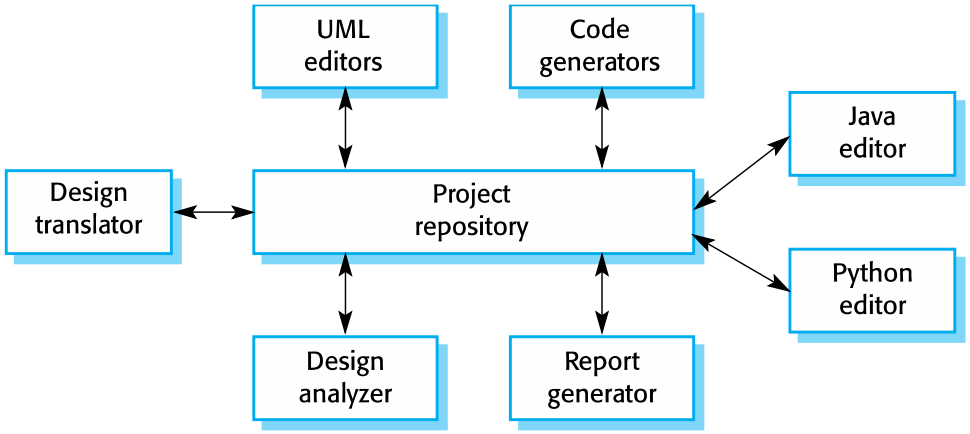
A generic layered architecture



Example: The architecture of the iLearn system

Repository architecture pattern

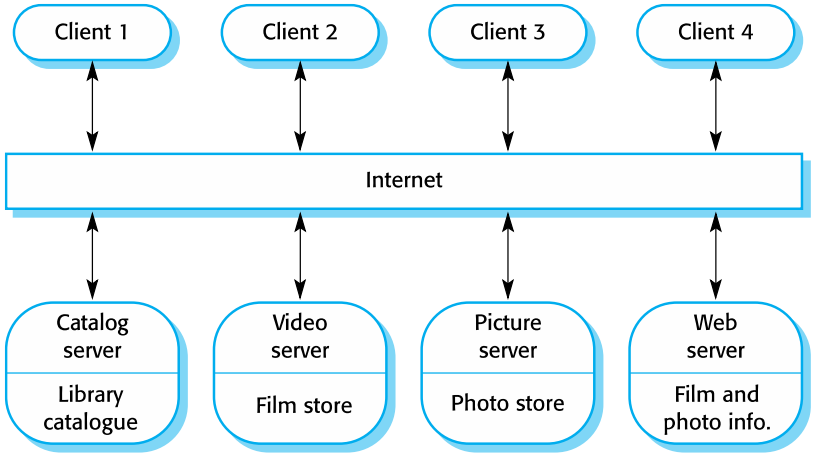
- **Description**: All data in a system is managed in a central repository. Components do not interact directly, only through the repository
- **When used**: Large volumes of information are generated that has to be stored for a long time / Data-driven systems where the inclusion of data triggers an action or tool.
- **Advantages**: Components can be independent. Changes made by one component can be propagated to all components. All data managed consistently in one place.
- **Disadvantages**: The repository is a single point of failure. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.



Example: A repository architecture for an IDE

Client–server architecture pattern

- **Description**: Functionality of the system is organized into services, with each service delivered from a separate server.
- **When used**: Data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
- **Advantages**: Servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients.
- **Disadvantages**: Each service is a single point of failure (e.g. by DDoS / server failure). Performance may be unpredictable depends on network. May be management problems if servers are owned by different organizations.



Example: A client–server architecture for a film library

SOLID Design Principles

What is SOLID?

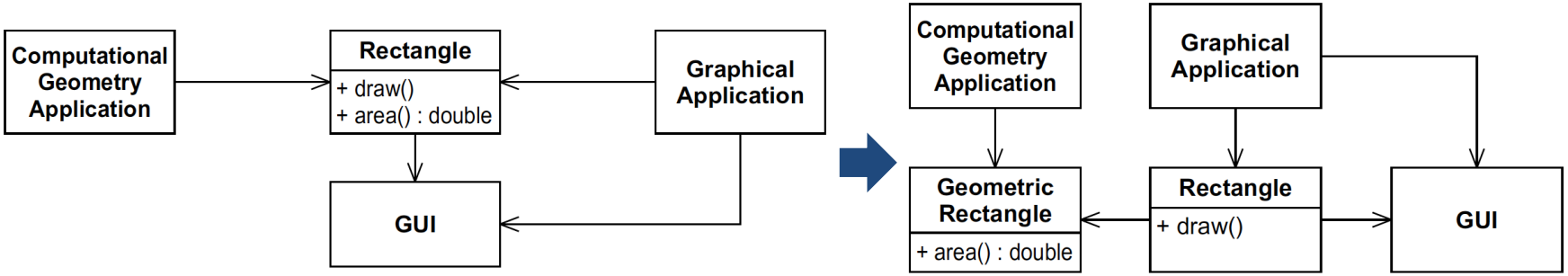
- ✧ **S**ingle Responsibility Principle (SRP)
- ✧ **O**pen Closed Principle (OCP)
- ✧ **L**iskov Substitution Principle (LSP)
- ✧ **I**nterface Segregation Principle (ISP)
- ✧ **D**ependency Inversion Principle (DIP)



Clean Code

Single Responsibility Principle (SRP)

- "A class should have **only one reason to change**."
  - A class should concentrate on doing one thing and one thing only.



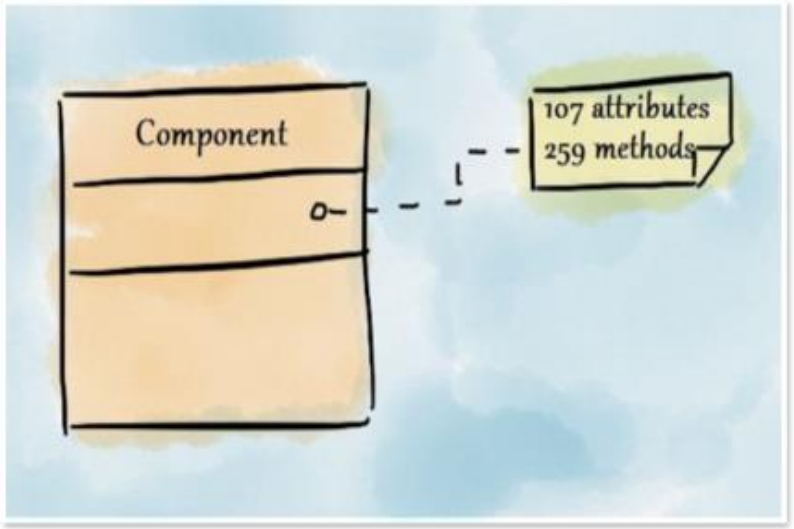
More than one responsibility

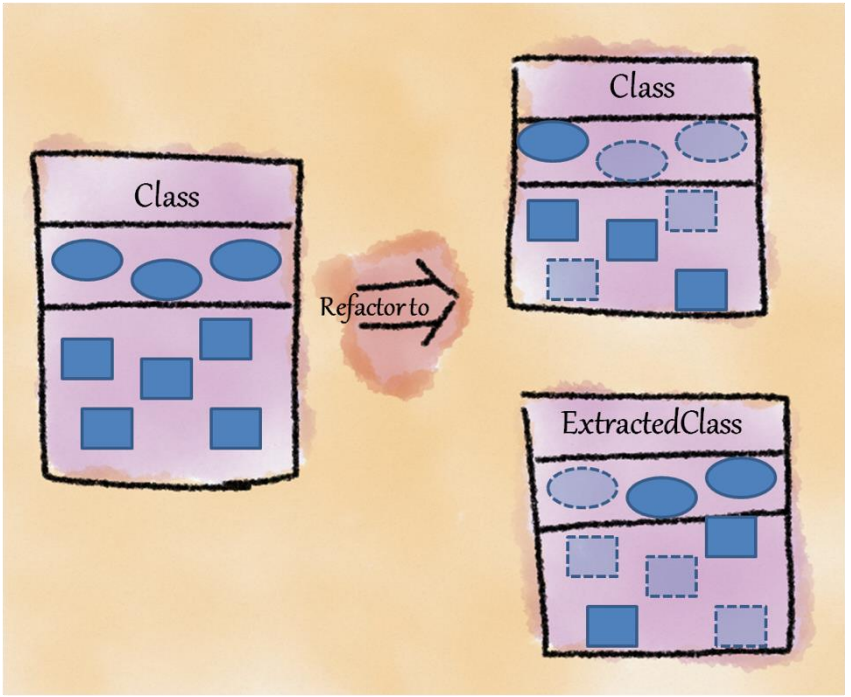
Separated Responsibilities

- Every time you create/change a class, you should ask yourself:

How many responsibilities does this class have?

Single Responsibility Principle (SRP)





(Source: Ganesh Samarthyam)

Examples

✧ BAD:

```
interface Modem {
 public void dial(String pno);
 public void hangup();
 public void send(char c);
 public char recv();
}
```

✧ Problems:

- Two responsibilities:
  - Connection Management
  - Data Communication

→ Smell of **Rigidity**

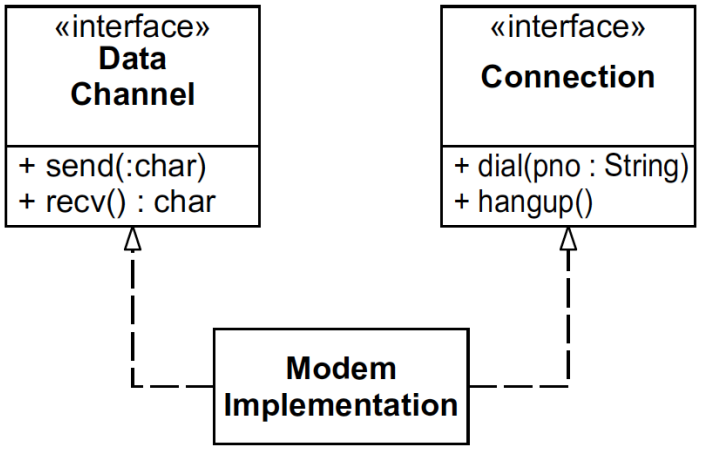
Examples

✧ GOOD:

- Separate into two interfaces

```
interface DataChannel {
 public void send(char c);
 public char recv();
}

interface Connection {
 public void dial(String pno);
 public void hangup();
}
```



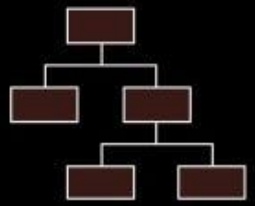
Caution!

If the application is not changing in ways that cause two responsibilities, no need to separate them. Otherwise, Smell of **Needless Complexity**

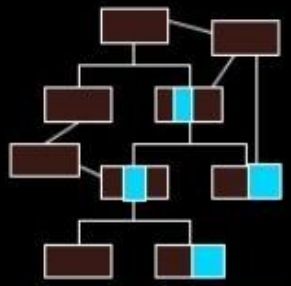
Open Closed Principle (OCP)

- "Software entities (classes, modules, functions, etc.) should be **open for extension, but closed for modification**."
  - If the OCP is applied well, then further changes of that kind are achieved by **adding new code**, **not by changing old code** that already works.
- **Idea**: build the classes in a way that we able to extend them by **child classes** and **inheritance.** Once you created the class, **it no longer needs to be changed.**

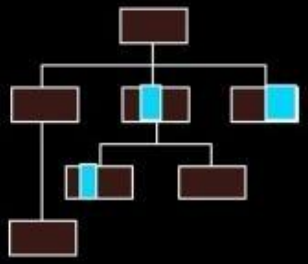
Usual way:



Starting code base

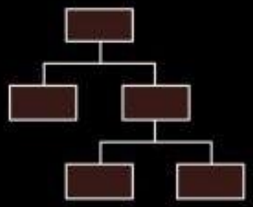


Changes implemented
blue == code changed

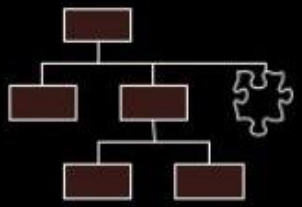


(Hopefully) Code cleaned up

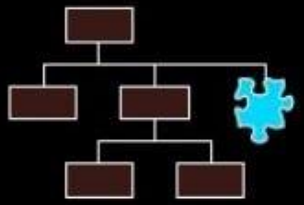
OCP:



Starting code base



Change design to make room for new feature



Implement feature

Source: Dave Nicolette

Examples

✧ **BAD**:

```
public enum PaymentType = { Cash, CreditCard };

public class PaymentManager{
 public PaymentType PaymentType { get; set; }
 public void Pay (Money money){
    if (PaymentType == PaymentType.Cash){
        //some code here - pay with cash
    }else{
        //some code here - pay with credit card
    }
 }
}
```

✧ **Problems**:

- Impossible to add a new `Payment`, e.g. Check, without modifying `PaymentManager`.
- Adding a new `Payment` to such an application means hunting for every place that such switch statements(or if/else chains) exist and adding the new `Payment` to each.

Examples

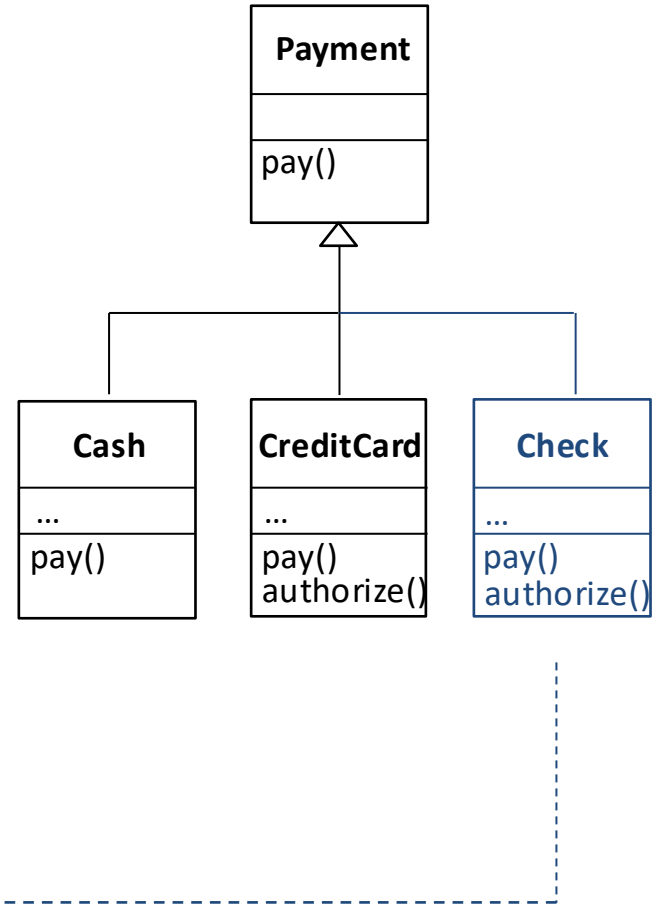
✧ GOOD:

- Open for extension:

```
public class Payment{
 public virtual void pay(Money money){
    // base class
 }
}
```

- Close for modification:

```
public class Cash:Payment{
 public override void pay(Money money){
    // pay with cash
 }
}
public class CreditCard:Payment{
 public override void pay(Money money){
    // pay with credit card
 }
}
```



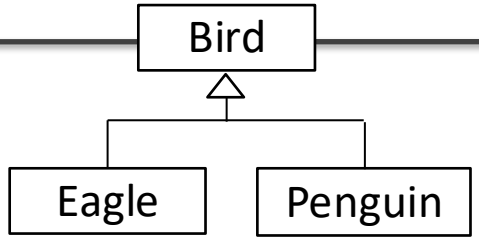
Liskov Substitution Principle (LSP)

- "If for each object o_1 of type S there is an object o_2 of type T such that for all programs P defined in terms of T, the behavior of P is unchanged when o_1 is substituted for o_2 then S is a subtype of T."



Subtypes Must Be Substitutable For Their Base Types.

Examples



✧ BAD:

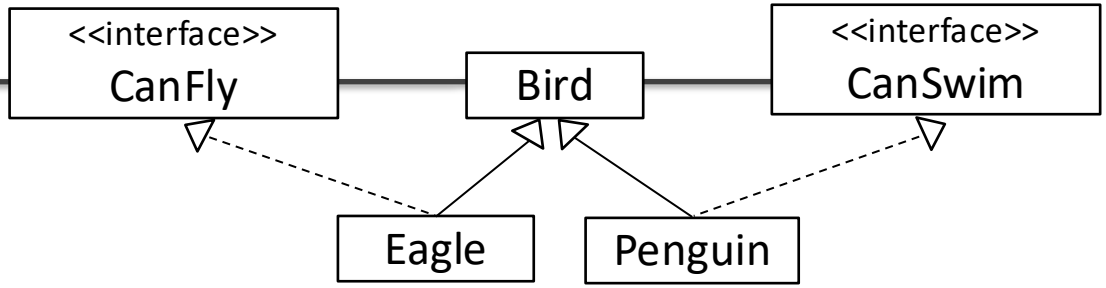
```
class Bird {
 let name: String
 let flySpeed: Double
 init(name: String, flySpeed: Double) {
    self.name = name
    self.flySpeed = flySpeed
 }
}
class Eagle: Bird { }
class Penguin: Bird {
 let swimSpeed: Double
 init(name: String, flySpeed: Double, swimSpeed: Double) {
    self.swimSpeed = swimSpeed
    super.init(name: name, flySpeed: flySpeed)
 }
}
let eagle = Eagle(name: "Eagle", flySpeed: 25)
// let penguin = Penguin(name: "Penguin",
//                       flySpeed: "can't fly", swimSpeed: 10)
```

✧ Problems:

- Penguin can't fly.
- Doesn't the IS-A relationship hold?

Examples

✧ **GOOD:**




```
protocol CanFly {
 var flySpeed: Double {
 get }
}
protocol CanSwim {
 var swimSpeed: Double {
 get}
}
class Bird {
 var name: String
 init (name: String) {
    self.name = name
 }
}
```

```
class Eagle: Bird, CanFly {
 let flySpeed: Double
 init(name: String, flySpeed: Double){
    self.flySpeed = flySpeed
    super.init(name: name)
 }
}
class Penguin: Bird, CanSwim {
 let swimSpeed: Double
 init(name: String, swimSpeed: Double){
    self.swimSpeed = swimSpeed
    super.init(name: name)
 }
}
```

Interface Segregation Principle (ISP)

- "Many specialized interfaces are better than one universal" *or*
- "Clients should not depend on methods that they do not use."

Examples

✧ BAD:

```
interface Worker {
 void work();
 void eat();
}
ManWorker implements Worker {
    void work() {…};
    void eat() {30 min break;};
}
RobotWorker implements Worker {
    void work() {…};
    void eat() {//Not Applicable
    for a RobotWorker};
}
```

✧ Problems:

- `RobotWorker` does not eat

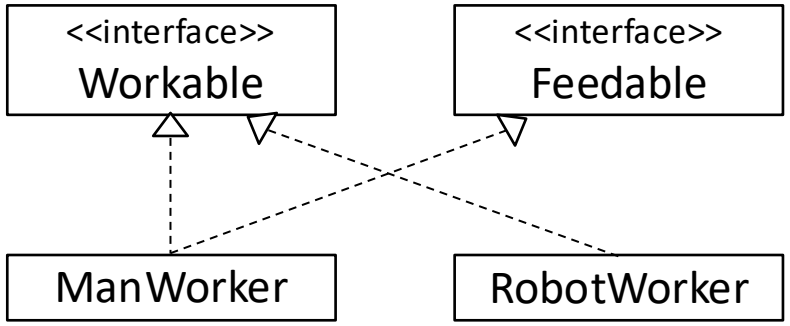
→ Smell of **Needless Complexity**

- `Worker` is a **Polluted Interface**

Examples

✧ GOOD:

- Split into two interfaces




```
interface Workable {
  void work();
}
interface Feedable {
  void eat();
}
ManWorker implements Workable, Feedable {
    void work() {…};
    void eat() {30 min break;};
}
RobotWorker implements Workable {
    void work() {…};
}
```

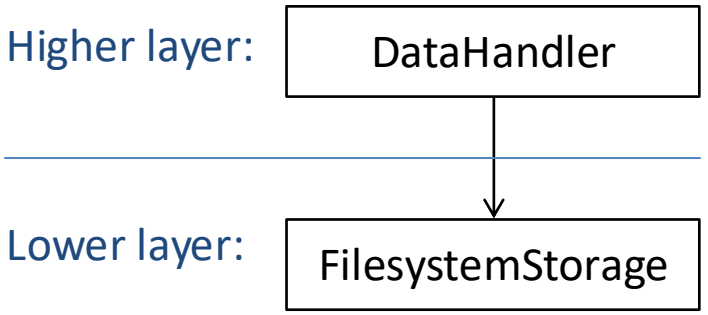
Dependency Inversion Principle (DIP)

- ✧ “High-level modules should not depend on low-level modules. Both should depend on abstractions.” and
- ✧ “Abstractions should not depend on details. Details should depend on abstractions.”

Examples

✧ BAD:

```
class FilesystemStorage {
 func save(_ data: Data) {
    // Create file->Save data locally
    print("Saving locally")
 }
}
class DataHandler {
 let storage = FilesystemStorage()
 func handle(_ data: Data) {
    storage.save(data)
 }
}
let handler = DataHandler()
handler.handle(Data())
```

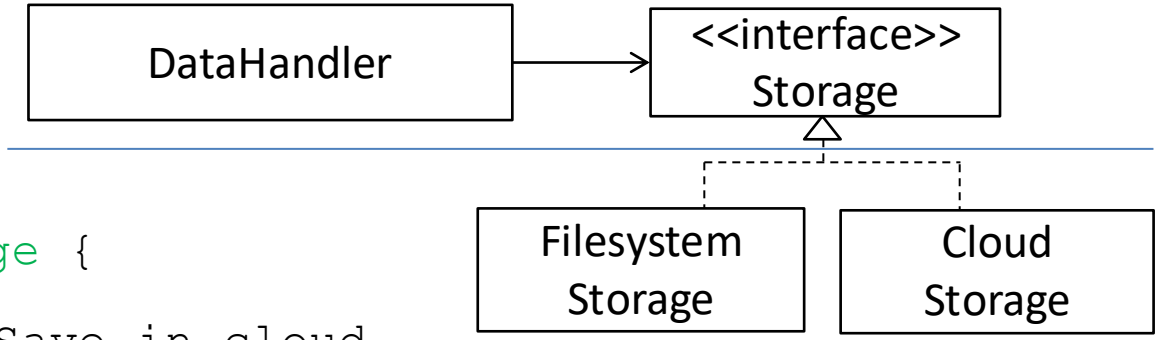


✧ Problems:

- DataHandler depends on FilesystemStorage. **Dependency is transitive.** DataHandler is **sensitive to changes all the way down**.

Examples

✧ GOOD:



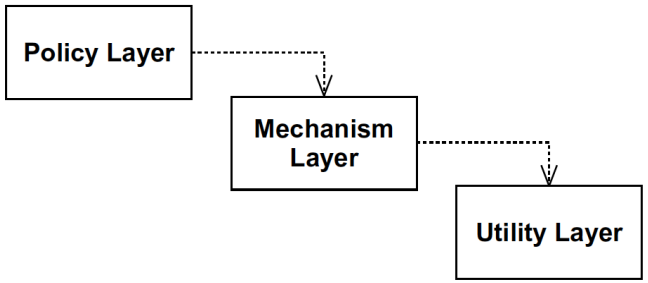

```
protocol Storage {
 func save(_ data: Data)
}
class CloudStorage: Storage {
 func save(_ data: Data) {
    // Connect to cloud > Save in cloud
    print("Saving in cloud")
 }
}
class FilesystemStorage: Storage {
 func save(_ data: Data) {
    // Create file > Save data locally
    print("Saving locally")
 }
}
```

```
class DataHandler {
 let storage: Storage
 init(storage: Storage){
    self.storage = storage
 }
 func handle(_ data: Data){
    storage.save(data)
 }
}
```

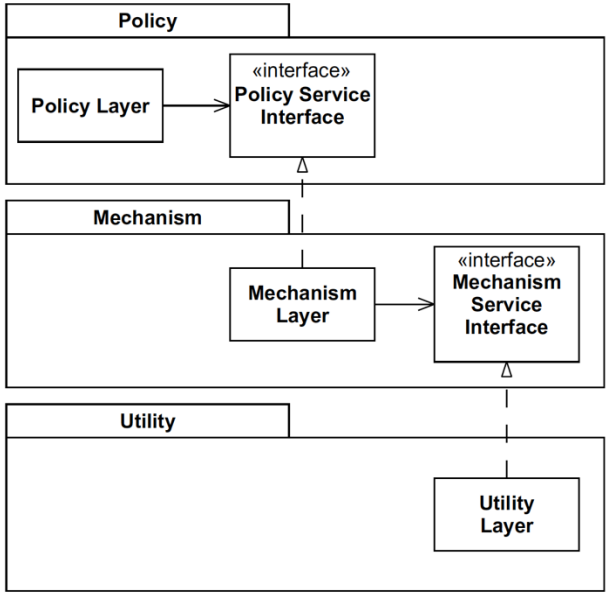
```
let storage = CloudStorage()
let handler = DataHandler(storage: storage)
handler.handle(Data())
```

Dependency Inversion Principle (DIP)

✧ Inversion of ownership:



Naive layering scheme



Inverted Layers

✧ Depend on Abstractions:

- No variable should hold a pointer or reference to a concrete class.
- No class should derive from a concrete class.
- No method should override an implemented method of any of its base classes.

Any Exception?

e.g. Java String

Summary for SOLID

Software inevitably changes/evolves over time (maintenance, upgrade)

- **S**ingle responsibility principle (SRP)
  - Every class should have only one reason to be changed
  - e.g. If class "A" has two responsibilities, create new classes "B" and "C" to handle each responsibility in isolation, and then compose "A" out of "B" and "C"
- **O**pen/closed principle (OCP)
  - Every class should be *open for extension* (derivative classes), but *closed for modification* (fixed interfaces)
  - e.g. Put the system parts that are likely to change into implementations (i.e. *concrete classes*) and define *interfaces* around the parts that are unlikely to change (e.g. *abstract base classes*)

Summary for SOLID

- **L**iskov substitution principle (LSP)
  - Subtypes must be substitutable for their base types.
  - Any functionality expected from the base type should be fulfilled by the subtype.
- **I**nterface segregation principle (ISP)
  - Keep interfaces as small as possible, to avoid unnecessary dependencies
  - Ideally, it should be possible to understand any part of the code in isolation, without needing to look up the rest of the system code
- **D**ependency inversion principle (DIP)
  - Instead of having concrete implementations communicate directly (and depend on each other), *decouple* them by formalizing their communication interface as an *abstract interface* based on the needs of the higher-level class

Why SOLID Matters More with AI

- AI writes code fast - structure is what keeps it testable and changeable.
- SRP keeps AI-generated classes small enough to review and unit-test.
- DIP is what makes code testable in isolation.
- OCP is what makes the project change request cheap: extend, don't rewrite (you may experience this after Client Meeting #2).